Rubric for Grading Assignments
MEC E 694: Applied Computational Intelligence for Engineers

Student Name: _____

Student Email: _____

Assignment #: _____

Coding Section

Level / Category	Contains at least 1 error (0 marks)	Can be Executed (3 marks)	Complete and Correct (5 marks)	Mark (0-5)	Weight	Total (%)
Sections 1, 2, 3, etc					×7.0	
35 points for all coding will be divided among the number of codes requested from you.						
General Comments						

Report Section

Level / Category	Inadequate (0-2 marks)	Adequate (3-4 marks)	Superior Work (5 marks)	Mark (0-5)	Weight	Total (%)
Results	Results simply stated.	Results stated & interpreted.	Results stated & interpreted & compared. Parameters discussed.		×7.0	
Discussion & Engineering Analysis	- Major errors in analysis. - Misunderstanding the concepts.	Minor errors in analysis.	No error in calculation and analysis.		×4.0	
Data Presentation (Figures, Tables, Symbols)	- Visuals & captions are poorly formatted. - Symbols are not defined.	Visuals & captions are formatted appropriately.	- Visuals & captions are formatted appropriately. - Symbols are defined.		×2.0	
					Total:	
General Comments						

Remarks:

1. Add this page to the beginning of your reports for grading (failing to attach the rubric will result in a mark deduction).
2. Submit one ZIP file containing your report and codes. The name of the ZIP file must be your last name.
3. Name your files correctly. For the report: one PDF file with your last name. For codes: all completed codes with names similar to the ones provided for you. If you should complete the file name "ABC," we only mark the codes in the file "ABC" and not any other file.

MEC E 694: Assignment #2

Due on Sunday, October 18, 2026

Nazarahari Fall 2026

Adam Jaikaran ID# 1708948

1. Feature Selection: Classical Methods

Code Files

`BestInd_d_Features_Incomplete.m`

2. Dimension Reduction, Principal Component Analysis (PCA)

Code Files

`KernelPCA_Incomplete.m`

Baseline

To establish a reference for both error and computational cost, the ANN was trained on the full BodyFat dataset (13 features, 252 observations) without feature selection or dimension reduction. All methods in Tables 1 and 2 were evaluated using a fitting network with 10 hidden neurons trained with Levenberg-Marquardt on a random 70/15/15 train/validation/test split, with the reported error being the mean of 0.5 $\times$ train MSE + 0.5 $\times$ test MSE over 50 independent runs. The baseline error was 23.69 (% body fat)^2, so any method whose error approaches this value has retained the information needed for prediction while using fewer inputs. It should be noted that the baseline time in Table 2 (3.9 s) is the cost of one complete ANN evaluation, whereas all other entries report only the cost of the feature selection or reduction step. Therefore, they are not directly comparable.

Sequential Forward/Backward Search

Since the number of selected features cannot be controlled for Sequential Forward/Backward Search, this evaluation was completed by setting the parameter $cv = 5$ and $cv = 10$ in the "sequentialfs" function resulting in 2 and 1 features being selected respectively (feature(s) [6,7] and [6]). The cv parameter sets the number of folds used to score each candidate subset. The data is split into k folds, the ANN is trained k times on k-1 folds and tested on the remaining fold, and the errors are accumulated. Thus cv does not determine the number of features directly, it changes how each subset is scored and therefore where the search stops. Sequential search was by far the most expensive method (112.1 s for cv = 10 and 83.4 s for cv = 5). This time intensive outcome is expected since the number of ANN trainings are proportional to a multiple of the cv parameter (250 trainings for $cv = 10$ and 180 for $cv = 5$, both $\approx 0.45s$ each).

Other Techniques

For the remaining methods, the parameter d sets the number of selected features after feature selection or dimension reduction, which is the number of original features retained for the feature selection methods, or the number of new components for the dimension reduction methods. Each method was evaluated for d = 1 to 5 using the same ANN protocol as the baseline. The methods fall into three groups. Laplacian Scores is a filter method. It ranks features without reference to the output or the classifier, by how well each feature preserves the neighbourhood structure of the data. Best Individual d Features is a wrapper method. It trains the ANN on each feature individually and keeps the d features with the lowest error. PCA and kernel PCA are feature extraction methods. Rather than keeping original features, they project the data onto d new components formed from combinations of all 13 features. Kernel PCA was evaluated with a linear kernel ($parameter = 1$, equivalent to PCA), a polynomial kernel ($degree = 2$), and a Gaussian kernel ($\sigma = 100$). See the Effect of Kernel Parameter section below for further parameter discussion.

Table 1: BodyFat dataset, weighted train/test MSE ((% body fat)^2) of the ANN, mean of 50 runs, for each method and number of features d.

	# of Features				
Method	**d = 1**	**d = 2**	**d = 3**	**d = 4**	**d = 5**
Baseline (all features included)			23.69		
Sequential Forward/Backward Search	25.84	30.96			
Laplacian Scores	50.60	51.46	30.41	21.41	21.64
Best Individual d Features	32.20	25.32	23.75	22.71	23.97
Principal Component Analysis	56.14	35.55	23.65	23.67	21.88
Linear Kernel PCA	67.04	39.75	25.89	32.09	24.64
Polynomial Kernel PCA	62.26	46.21	24.65	35.86	30.05
Gaussian Kernel PCA	39.19	34.57	34.65	34.57	26.03

Table 2: Computational Time (s) for Feature Selection and Extraction Methods

	# of Features				
Method	**d = 1**	**d = 2**	**d = 3**	**d = 4**	**d = 5**
Baseline (all features included)			3.9		
Sequential Forward/Backward Search	112.1	83.40			
Laplacian Scores	0.018	0.002	0.002	0.002	0.003
Best Individual d Features	15.00	14.74	16.02	16.54	14.77
Principal Component Analysis	0.005	0.001	0.001	0.001	0.001
Linear Kernel PCA	0.012	0.004	0.004	0.005	0.003
Polynomial Kernel PCA	0.014	0.016	0.012	0.016	0.012
Gaussian Kernel PCA	0.012	0.014	0.011	0.014	0.013

Method Comparison (Error)

Because each error is the mean of randomly initialized ANN runs, small differences between cells are not meaningful. For example, linear KPCA and PCA are mathematically equivalent yet they differ by up to 8.4 ($d = 4$). Because of this, results are interpreted in three approximate categories: good ($\sim$ 21–27, comparable to the baseline of 23.69), moderate ($\sim$ 30–40), and poor ($\sim$ 46–68). Every method qualifies as 'good' for at least one value of d. 'Poor' outcomes only occur at d = 1–2 and only for the unsupervised methods (Laplacian Scores, PCA and KPCA). Note that the two wrapper methods never exceed 32.2, because they select features using the ANN's own error.

Now going through each method individually. Sequential search gave the best small-d result, 25.84 with a single feature ($cv = 10$), within about 2 of the baseline. The 2-feature subset ($cv = 5$) scored worse (30.96). Laplacian Scores was poor at $d = 1 - 2$ (about 51), then reduced at $d = 3$ (30.4), and gave the lowest errors overall at $d = 4 - 5$ $(21.4, 21.6)$. Since Laplacian Scores is an unsupervised filter, its best features preserve the data's neighbourhood structure but are not the most predictive of body fat. Best Individual d Features was the most consistent method, moderate only at $d = 1$ (32.2) and in the good band from $d = 2$ for the rest. Ranking by ANN error puts predictive features first. The plateau near 23 likely reflects redundancy, since the top individual features are related body measurements that add little new information. PCA was poor at $d = 1$ (56.1) and moderate at $d = 2$ (35.6), then matched the baseline from $d = 3$ $(23.7, 23.7, 21.9)$. Since the first two components capture maximum variance, which on unscaled data is dominated by large

value features, they may not be as aligned with body fat percentage. For example the variance in weight (lbs) is a much larger than, for example the variance in wrist circumference (cm). Linear KPCA followed the same trend as PCA ($67.0 \rightarrow 39.8 \rightarrow 25.9$), as expected for an equivalent method. Its differences likely come from run-to-run noise. Polynomial KPCA matched linear at $d = 3$ (24.7) but was poor at $d = 1 - 2$ $(62.3, 46.2)$ and inconsistent at $d = 4 - 5$ $(35.9, 30.1)$. Gaussian KPCA was the best extraction method at $d = 1$ (39.2) but plateaued near 34.6 for $d = 2 - 4$, reaching the good band only at $d = 5$ (26.0).

Method Comparison (Time)

The two wrapper methods were orders of magnitude slower than all other methods (Table 2), because each candidate feature set is scored by training the ANN. Best Individual d Features required ~ 15 s regardless of d, consistent with 13 features $\times$ 20 ANN runs = 260 trainings at ~ 0.06 s each. This is comparable to the run cost implied by the baseline (3.9 s / 50 $runs \approx 0.08$ s). Sequential search was the slowest (83–112 s), since it scores many subsets and each requires one ANN training per cross validation fold. The filter and extraction methods required only milliseconds because they never train the ANN. PCA was the fastest (~ 0.001 s). Laplacian Scores was comparable (~ 0.002 s). Kernel PCA took about $10\times$ longer, because it builds and decomposes a 252×252 kernel matrix, so its cost grows with the number of observations rather than features. For all methods, time was essentially independent of d. Each method computes its full ranking or decomposition once and then keeps the first d features or components. The higher times at $d = 1$ likely reflect one-time MATLAB code loading on the first call and are not a property of the methods.

Effect of Feature Set Size on PCA

For PCA, the cumulative explained variance measures how much of the original data's variance is retained after projecting it onto the first d principal component(s). More simply, it is a measure of how much information is retained after reducing the number of components. Generally, as the number of components increases, the cumulative variance increases and the error decreases with two notable caveats. The first being the plateau observed at $d = 3$. Once the third component is introduced, the error plateaus near the baseline level (23.65) indicating little useful information is added by further components. Though a small increase in cumulative variance is observed from $d = 3$ (97.3%) to $d = 5$ (98.7%) no meaningful decrease in error follows, within the noise of the algorithm. The second caveat is the strength of correlation between the cumulative variance and the PCA error. As noted, a small increase in cumulative variance from $d = 3$ to $d = 5$ led to negligible decrease in PCA error. But, for $d = 2$ to $d = 3$ a similar increase in cumulative variance lead to an error reduction from 35.6 to 23.7. This shows that the variance retained is not equivalent to predictive information retained. PCA is unsupervised and ranks component directions by spread, not by relevance to body fat. This likely led to the first principal component being dominated by the largely varying weight measurement and the information most relevant to body fat lying in a lower varying component direction. This method could be improved by standardizing the features before performing PCA so that large variances due to measurement unit choice does not have as great an effect.

Table 3: PCA error ((% body fat)^2) vs. cumulative explained variance, BodyFat dataset

d	Cumulative Variance	PCA Error
1	82.8%	56.14
2	95.6%	35.55
3	97.3%	23.65
4	98.1%	23.67
5	98.7%	21.88

Kernel Function Performance and Effect of Kernel Parameter

To examine the effect of kernel parameter, each kernel was evaluated at $d = 3$ for varying kernel parameter values (Figure 1). Due to the difference in mathematical operations for each kernel, these parameters were varied within each function's valid range. For example, the power and log kernels (exponential kernel parameters were varied between 0 and 2). Whereas the distance parameters such as in the gaussian kernel were tested at $0.1\times, 1\times, and\ 10\times$ the median pairwise distance between observations. Each kernel was then re-evaluated at its best tested parameter over five random seeds (Figure 2).

The best performing kernels were those that reduced to linear or near linear functions. These include the polynomial kernel with degree 2 (28.46 ± 8.80), the hyperbolic tangent kernel for small α (24.35 ± 1.88), and the power kernel with exponent 2 (26.60 ± 3.52). Each performed within the noise of linear KPCA and the baseline. This result is expected as at a small α or slope, *tanh* operates in its approximately linear region and the power kernel with exponent 2 mathematically simplifies to linear. In contrast, the distance based kernels (Gaussian, Laplacian, Rational Quadratic, and Log) plateaued at higher error ($\approx 33 - 42$) and the Multiquadric kernel performed worst. Together these results suggest that body fat is close to a linear function of the data set measurements, thus kernels that introduce nonlinearity performed worse.

The kernel parameter mathematically affects the behaviour of the kernel function, and in this case, determines how far each kernel departs from its near linear regime. For Gaussian and Laplacian kernels, a small parameter value results in large errors whereas increasing the parameter progressively approaches linear behaviour resulting in a reduced error. The hyperbolic tangent kernel showed the opposite behaviour with respect to kernel parameter. For large values of kernel parameter the tanh kernel showed large error, in contrast with lesser error for small values of kernel parameter. For the polynomial kernel, increasing the degree beyond 2 raised the error because high powers of the dot product term allow a few extreme observations to dominate the kernel matrix. The log kernel was nearly unaffected by its exponent since pairwise distances are much larger than 1, so the exponent only rescales the kernel. These conclusions are limited by the run-to-run variability of the ANN and the use of unstandardized features, which allows large-valued measurements to dominate all distance and dot-product computations.

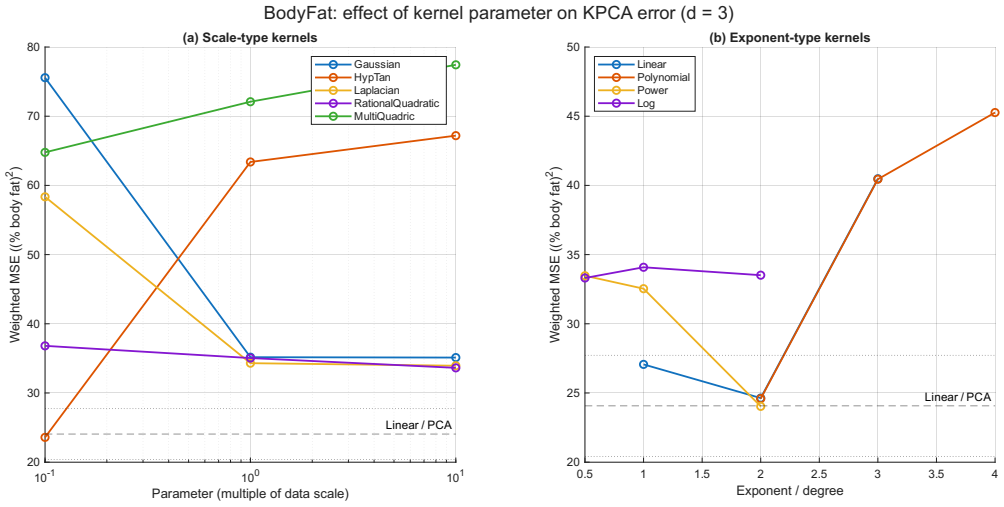


Figure 1: Effect of kernel parameter on KPCA error (BodyFat, d = 3): (a) scale-type kernels at $0.1\times$, $1\times$, $10\times$ the data scale, (b) exponent-type kernels. Dashed line: linear/PCA reference.

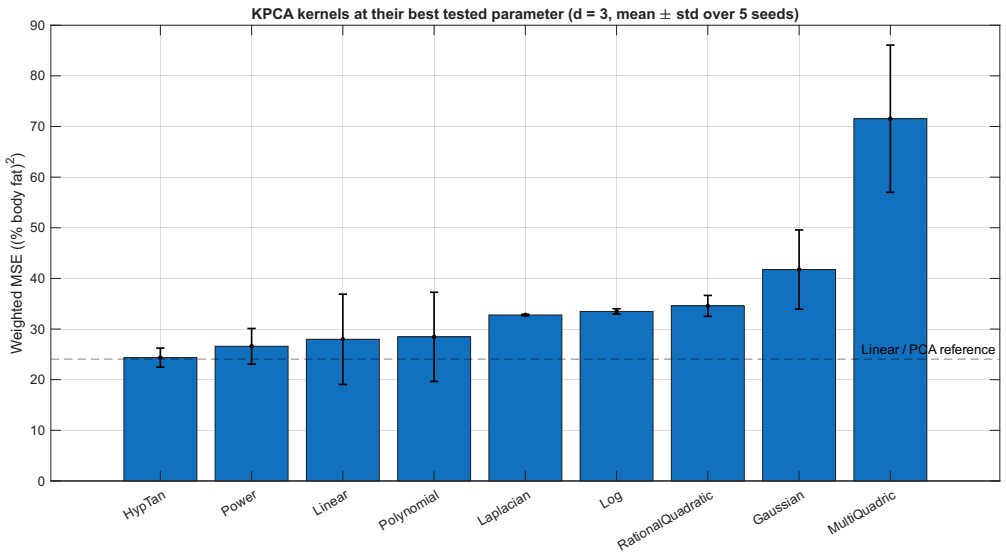


Figure 2: Comparison of KPCA Function Performance, Each kernel at its best tested parameter, d = 3, mean ± std over 5 seeds; dashed line: linear/PCA reference.

Overfitting

Overfitting occurs when a model learns noise specific to its training data rather the true underlying relationship, resulting in low training error but high error when faced with new data. The risk of overfitting increases with the number of input features. Each additional input adds parameters to the ANN and each additional parameter gives the ANN more degrees of freedom to fit the noise in data. For this data set specifically, many measurements are strongly correlated which adds parameters without adding information, which can cause the peaking phenomenon discussed in lecture. Beyond a certain point additional features begin to degrade performance. Thus the trade-off comes in the form of balancing overfitting, where too many features cause the model to fit noise, and underfitting, where too few features miss useful information (such as PCA at $d = 1$). This is where the feature selection and dimension reduction come in. By removing redundant inputs or combining correlated features, both methods reduce the number of parameters the ANN must estimate. This improvement is exemplified by some methods beating the error score of the 13-feature baseline using only $3 - 5$ inputs (such as Laplacian and PCA). That being said the marginal improvement shown is likely within the run-to-run variation so the baseline, in this example, does not show strong signs of overfitting.

Best and Worst Dimension Reduction Methods for d = 2

Figure 3 shows the plotted 2-dimensional data for the best (Best Individual d Features, error = 25.32) and worst (Laplacian Scores, error = 51.46) methods. The colouring depicts the corresponding body fat percentage data to show the ability of the methods to sort the data. For Best Individual d Features, the selected features (Abdomen and Chest) show a clear gradient in body fat. For Laplacian Scores, the selected features (Hip and Weight) show high and low body-fat observations overlapping throughout the plot. This difference is reflected in the way each method selects features. Best Individual d Features is a wrapper that ranks features by the ANN's own error, whereas Laplacian Scores is an unsupervised filter that favors features preserving the local neighbourhood structure of the data, with no reference to body fat. Therefore, its top ranked features are not necessarily the most predictive of body-fat percentage, which explains the poor performance until more features are included.

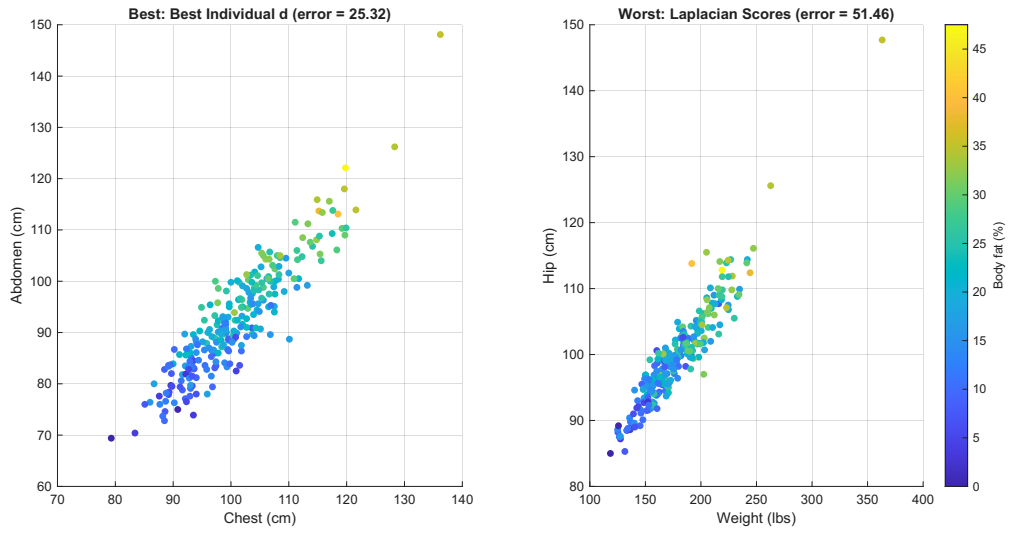


Figure 3: Best and Worst Methods for d = 2, points coloured by body fat (%)

3. Evolutionary Feature Selection with NSGA-II (Optional)

Evolutionary Feature Selection with NSGA-II is not included in this report.

4. Building a Feature Selection & Dimension Reduction Pipeline

Code Files

`HAR_Pipeline.m`
`LoadHAR.m`
`RunKPCA.m`
`NeighborhoodPreservation.m`
`PlotEmbedding2D.m`
`PlotEmbedding3D.m`

Pipeline Overview

The pipeline was implemented in MATLAB as a single main script, `HAR_Pipeline.m`, which runs all stages in order and can be executed top to bottom. All settings are defined in one configuration block at the top of the script. The random seed is fixed (`Config.Seed = 1`) so that the subsample and all embeddings are reproducible. The pipeline consists of five stages:

1. **Data ingestion and sanity checks** (`LoadHAR.m`): loads and combines the train and test partitions, then reports the number of samples and features, data types, missing values, value range, constant features, outliers and the label distribution.

2. **Preprocessing**: standardizes all features to zero mean and unit variance and saves the transformation parameters.

3. **Feature selection**: ranks the original features by Laplacian Score (unsupervised).

4. **Dimension reduction**: PCA (fit on the full dataset), Kernel PCA with Gaussian and Laplacian kernels (`RunKPCA.m`), and t-SNE as an additional nonlinear comparison.

5. **Analysis and visualization**: 2D and 3D embeddings coloured by activity (`PlotEmbedding2D.m`, `PlotEmbedding3D.m`), and a quantitative comparison of neighbourhood preservation (`NeighborhoodPreservation.m`) and runtime.

Load & check → Standardize → Laplacian Score → PCA / KPCA / t-SNE → Plots & metrics

Data Ingestion & Sanity Checks

The training and test sections of the UCI HAR dataset were combined into a single set of 10 299 samples and 561 continuous features. All features are numeric, with no missing values and no constant features. Additionally, all values lie in the range of $[-1, 1]$ indicating the features were already normalized. The breakdown of the six activities are listed in Table 4. A median absolute deviation rule (3 scaled MADs) flagged 25% of entries as outliers. However, this likely reflects the bimodal distribution of many features due to activities like laying and walking occupy different ranges, so these values were kept.

Table 4: UCI HAR activity distribution (combined train + test, n = 10,299)

Activity	Count	Percent (%)
WALKING	1722	16.7
WALKING UPSTAIRS	1544	15.0
WALKING DOWNSTAIRS	1406	13.7
SITTING	1777	17.3
STANDING	1906	18.5
LAYING	1944	18.9

Preprocessing

No missing values were present. Even though all features were already scaled to $[-1, 1]$, their standard deviations still ranged from 0.037 to 0.750, a 20× difference, corresponding with a 400× difference in variance. Since PCA aims to maximize variance, the unscaled data would be dominated by a subset of highly varying features. So, each feature was standardized to zero mean and standard deviation equal to 1.

Feature Selection (Laplacian Score)

The Laplacian Score ranks features by how well they preserve the local neighbourhood structure of the data, notably without using labels. All ten top-ranked features measure overall motion intensity (signal magnitude area and acceleration magnitude), reflecting what dominates structure in the data. Namely, the separation between dynamic and static activities. These features are highly correlated, illustrating a limitation of this filter, which cannot account for redundancy among the selected features.

Dimensionality Reduction

Figures 4 and 5 show the 2D and 3D visualization of a 3 000 point subsample, coloured by activity. Noting that labels were used for visualization colouring only.

Laplacian Score - The top two ranked features (tBodyAcc-sma and tBodyAccJerk-sma) lie along a narrow diagonal band, confirming that they are highly correlated and redundant. They separate the static activities from the dynamics ones but fails to strongly differentiate the static activities from one another (laying vs. sitting). This behaviour is expected due to the method of Laplacian scores rating each feature

independently by how well it preserves local neighbourhood structure. Because the dominant structure in the dataset is the static/dynamic split, the top features capture it and the second feature adds little new information.

PCA - The first principal component is essentially a static vs. dynamic axis. Static activities are negative while dynamic activities are positive. This is expected since PCA orders components by variance, and as established, the largest variance occurs in the direction differing moving and not moving. In 2D (Figure 4) the static activities are not clearly separated, however, in 3D (Figure 5) though they still remain connected, distinct bands begin to appear. This suggests the third component carries some posture information. It should also be noted that a few of the walking downstairs samples have extreme component 1 values (near 100), likely reflecting high speed or intensity movements. With only 3 components, the variance retained reaches only 59.7% (65 components needed for 90%), so much of the finer structure is not resolved.

Kernel PCA - The Gaussian and Laplacian kernels produce similar results to one another, as expected since both define similarity from distance related terms. Compared with PCA, the static activities are much more spread out rather than compressed into a small region. Performing a kernel parameter sweep, as kernel parameter increased, the Gaussian kernel progressively approached the PCA behaviour, consistent with the kernel approximating a linear kernel at large kernel parameters. Polynomial Kernels were also tested but were dominated by a few extreme observations, which compressed the remaining points and were therefore not used.

t-SNE - The t-SNE produced the clearest grouping, with three main clusters (dynamic activities, sitting/standing, and laying), with finer sub-groupings by activity also visible. Interestingly, laying down forms a clear separate cluster. These results likely reflect sensing the changes in the phone's orientation relative to gravity. Since t-SNE explicitly optimizes the preservation of local neighbourhoods, the clusters are most clear, however, distances between clusters and relative cluster sizes are not meaningful, because t-SNE optimizes only the preservation of local neighbourhoods. The result also depends on the perplexity parameter and random initialization.

Analysis & Visualization Outputs

Table 5 reports the neighbourhood preservation (fraction of each point's 10 nearest neighbors to the original 561–dimensional space that remain among its 10 nearest neighbors) and runtime on the subsample. All methods exceed the random embedding baseline (0.003). t-SNE preserved the most local structure (0.461). PCA (0.131) outperformed both KPCA kernels (0.087–0.093) and the Laplacian Score features (0.078). This differs however from the plot visuals. The KPCA method spreads the static activities more clearly yet preserve fewer near neighbors. This suggests that neighbor preservation and cluster separation measure different aspects of clustering. All methods ran approximately under 3 seconds for 3000 samples. Since KPCA requires an $n \times n$ kernel matrix, the cost grows rapidly with number of samples, thus a smaller, 3000 sample subset was used for comparison. This cost contrasts with the PCA method as its cost depends mainly on the number of features.

Looking at the plots, the Laplacian Score is interpretable, as it retains physical features, but its ranking method selects a second redundant feature causing poor structure preservation. PCA is fast and interpretable through explained variance, but primarily captures only a linear structure dominated by the largest source of variance. KPCA can reveal nonlinear structure, but it requires choosing a kernel and kernel parameter and its memory cost can be significant. t-SNE gives the clearest visual clusters and the best neighbourhood preservation, however the global distances and geometry of the plot are not meaningful and like KPCA can be sensitive to its parameters.

Limitations Results are based on a balanced subsample of 3,000 points and a single random seed, KPCA parameters were selected visually over a coarse grid, and all comparisons use only three dimensions.

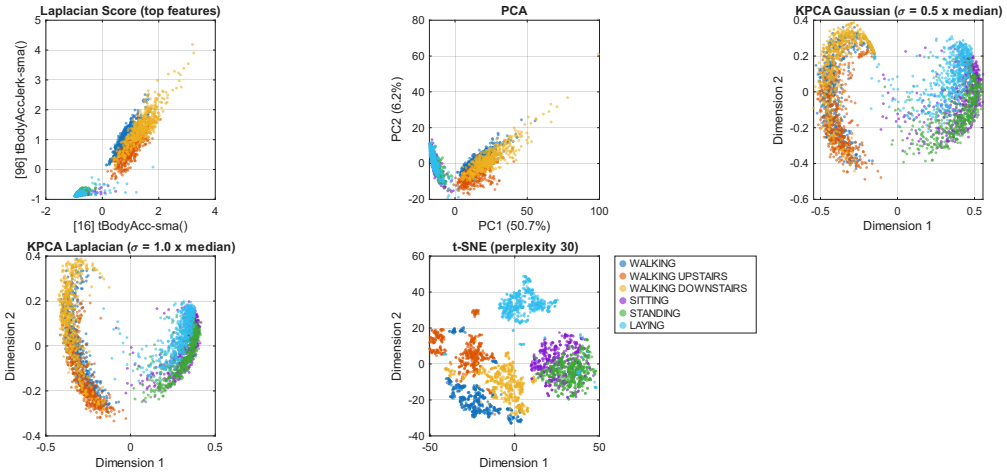


Figure 4: 2-Dimensional Method Comparison of balanced 3,000-point subsample of UCI HAR

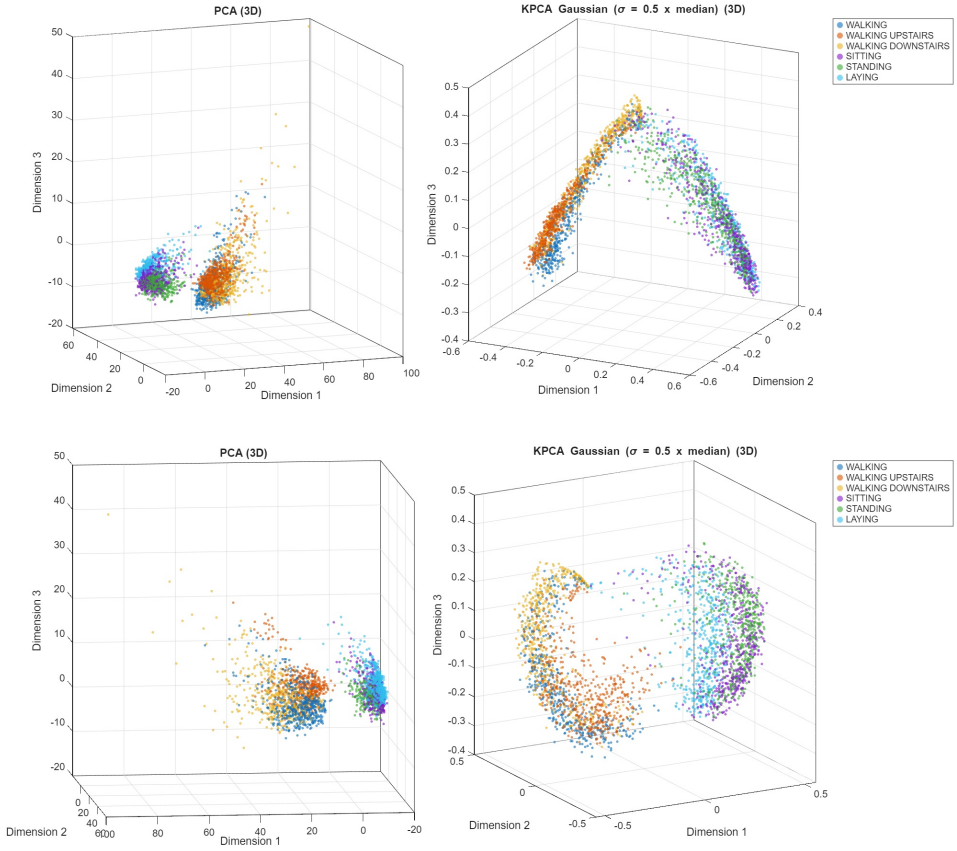


Figure 5: 3-Dimensional Method Comparison of balanced 3,000-point subsample of UCI HAR, at two viewing angles

Table 5: Neighbor Preservation and Runtime Comparison

Method	**Neighbor Preservation**	**Runtime (s)**
Laplacian Score	0.08	0.37
PCA	0.13	0.10
KPCA Gaussian	0.09	0.25
KPCA Laplacian	0.09	0.21
t-SNE	0.46	3.07